

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc. I.T Semester IV)
Artificial Intelligence

Practical - 1

Roll No.: TY18	Name: Sachin Ramesh Mehta
Class: TYIT	Batch: 1
Date of Assignment: 27/06/2026	Date/Time of Submission: 01/07/2026

Aim:- Design a Simple Rational Agent by defining PEAS for Vacuum Cleaner Environment and Autonomous Taxi. Classify environments (fully/partially observable, deterministic/stochastic, episodic/ sequential). Implement a simple table-driven agent in Python.

Vacuum Cleaner :

Code :-

```
table = {
    ('A', 'Dirty'): 'Suck',
    ('A', 'Clean'): 'Right',
    ('B', 'Dirty'): 'Suck',
    ('B', 'Clean'): 'Left',
    ('C', 'Dirty'): 'Suck',
    ('C', 'Clean'): 'Right',
    ('D', 'Dirty'): 'Suck',
    ('D', 'Clean'): 'Right'
}

sequence_table = {
    (('A', 'Clean'), ('B', 'Clean'), ('C', 'Clean'), ('D', 'Dirty')): 'Right',
    (('A', 'Clean'), ('B', 'Clean'), ('C', 'Dirty'), ('D', 'Clean')): 'Suck',
    (('A', 'Clean'), ('B', 'Dirty'), ('C', 'Clean'), ('D', 'Clean')): 'Left',
    (('A', 'Dirty'), ('B', 'Clean'), ('C', 'Clean'), ('D', 'Clean')): 'Suck',
    (('A', 'Dirty'), ('B', 'Dirty'), ('C', 'Dirty'), ('D', 'Clean')): 'Left',
    (('A', 'Dirty'), ('B', 'Dirty'), ('C', 'Clean'), ('D', 'Dirty')): 'Suck',
    (('A', 'Dirty'), ('B', 'Clean'), ('C', 'Dirty'), ('D', 'Dirty')): 'Left',
    (('A', 'Clean'), ('B', 'Dirty'), ('C', 'Dirty'), ('D', 'Dirty')): 'Suck'
}
```

```

percepts = []

def table_driven_agent(percept):
    percepts.append(percept)

    if tuple(percepts) in sequence_table:
        return sequence_table[tuple(percepts)]

    return table.get(percept, "NoOp")

print("Vacuum Cleaner Table-Driven Agent")
print("Sachin Ramesh Mehta, TY18")

n = int(input("Enter number of percepts: "))

for i in range(n):
    print(f"\nPercept {i+1}")

    location = input("Enter Location (A/B/C/D): ").upper()
    status = input("Enter Status (Clean/Dirty): ").capitalize()

    percept = (location, status)

    action = table_driven_agent(percept)

    print("Percept History :", percepts)
    print("Agent Action  :", action)

```

Output :-

```
Vacuum Cleaner Table-Driven Agent  
Sachin Ramesh Mehta, TY18
```

```
Percept 1  
Percept History : [('A', 'Clean')]  
Agent Action    : Right
```

```
Percept 2  
Percept History : [('A', 'Clean'), ('B', 'Dirty')]  
Agent Action    : Suck
```

```
Percept 3  
Percept History : [('A', 'Clean'), ('B', 'Dirty'), ('C ', 'Clean')]  
Agent Action    : NoOp
```

```
Percept 4  
Percept History : [('A', 'Clean'), ('B', 'Dirty'), ('C ', 'Clean'), ('D ', 'Dirty')]  
Agent Action    : NoOp
```

Autonomous Taxi

Code :-

```
table = {  
    ('Green', 'No', 'No', 'No'): 'Move Forward',  
    ('Green', 'Yes', 'No', 'No'): 'Stop',  
    ('Red', 'No', 'No', 'No'): 'Stop',  
    ('Yellow', 'No', 'No', 'No'): 'Slow Down',  
    ('Green', 'No', 'Yes', 'No'): 'Pick Up Passenger',  
    ('Red', 'Yes', 'No', 'No'): 'Stop',  
    ('Yellow', 'Yes', 'No', 'No'): 'Stop',  
    ('Green', 'No', 'No', 'Yes'): 'Wait for cross'  
}  
  
percepts = []  
  
def taxi_agent(percept):  
    percepts.append(percept)  
  
    action = table.get(percept, "Wait")  
  
    return action  
  
print("Autonomous Taxi Table-Driven Agent")  
print("Sachin Ramesh Mehta, TY18")  
  
n = int(input("Enter number of percepts: "))  
  
for i in range(n):
```

```

print(f"\nPercept {i+1}")

signal = input("Traffic Signal (Green/Yellow/Red): ").capitalize()
obstacle = input("Obstacle Present? (Yes/No): ").capitalize()
passenger = input("Passenger Request? (Yes/No): ").capitalize()
crossing = input("Indigator any? (Yes/No): ").capitalize()
percept = (signal, obstacle, passenger, crossing)

action = taxi_agent(percept)

print("Current Percept:", percept)
print("Agent Action:", action)

```

Output :-

```

Autonomous Taxi Table-Driven Agent
Sachin Ramesh Mehta, TY18

Percept 1
Current Percept: ('Green', 'Yes', 'Yes', 'Yes')
Agent Action: Wait

Percept 2
Current Percept: ('Yellow', 'No', 'No', 'No')
Agent Action: SLOW DOWN

Percept 3
Current Percept: ('Red', 'Yes', 'No', 'Yes')
Agent Action: Wait

```

Automatic-Door

Code :-

```

table = {
    ('Closed', 'Known', 'No', 'Correct'): 'Open Door',
    ('Closed', 'Known', 'No', 'Wrong'): 'Deny Access',
    ('Closed', 'Unknown', 'No', 'Any'): 'Ask for ID',
    ('Closed', 'Known', 'Yes', 'Any'): 'Sound Alarm',
    ('Closed', 'Unknown', 'Yes', 'Any'): 'Sound Alarm',

    ('Open', 'Known', 'No', 'Correct'): 'Allow Entry',
    ('Open', 'Known', 'No', 'Wrong'): 'Deny Access',
    ('Open', 'Unknown', 'No', 'Any'): 'Ask for ID',
    ('Open', 'Known', 'Yes', 'Any'): 'Close Door',

```

```

    ('Open', 'Unknown', 'Yes', 'Any'): 'Sound Alarm'
}

percepts = []

def security_agent(percept):
    percepts.append(percept)
    return table.get(percept, "Wait")

print("Smart Door Security Table-Driven Agent")
print("Sachin Ramesh Mehta, TY18")

n = int(input("Enter number of percepts: "))

for i in range(n):
    print(f"\nPercept {i+1}")

    door = input("Door Status (Open/Closed): ").capitalize().strip()
    person = input("Person (Known/Unknown): ").capitalize().strip()
    emergency = input("Emergency (Yes/No): ").capitalize().strip()
    password = input("Password (Correct/Wrong/Any): ").capitalize().strip()

    percept = (door, person, emergency, password)
    action = security_agent(percept)

    print("Percept History:", percepts)
    print("Agent Action:", action)

```

Output :-

```

Smart Door Security Table-Driven Agent
Sachin Ramesh Mehta, TY18

Percept 1
Percept History: [('Open', 'Known', 'No', 'Khul jaa sim sim')]
Agent Action: Wait

Percept 2
Percept History: [('Open', 'Known', 'No', 'Khul jaa sim sim'), ('Closed', 'Unknown', 'Yes', 'Jack')]
Agent Action: Wait

Percept 3
Percept History: [('Open', 'Known', 'No', 'Khul jaa sim sim'), ('Closed', 'Unknown', 'Yes', 'Jack'),
Agent Action: Deny Access

```